

Chapter 1

8086 Architecture

Learning Outcomes

Upon completing this chapter the reader will know the architecture of 8086 with special attention to the followings:

- ❖ The essential functional components of 8086
- ❖ General registers and their functions
- ❖ Special registers and their functions
- ❖ Flag register containing status flags and their functions
- ❖ Addressing mechanism and memory segmentation in 8086
- ❖ Physical and logical addressing
- ❖ Pipelining
- ❖ Interrupt structure and their functionality

1.1 Basic Computer System

A basic computer system is shown in Fig. 1.1. The core of the computer system is the central processing unit (CPU) where all the arithmetic, logical and data movements are controlled.

The CPU needs program memory and data memory for essential computations, while, visual devices like display monitor and input devices like keyboards are required for operator interaction. The CPU manages communications with all devices in its periphery through a system bus.

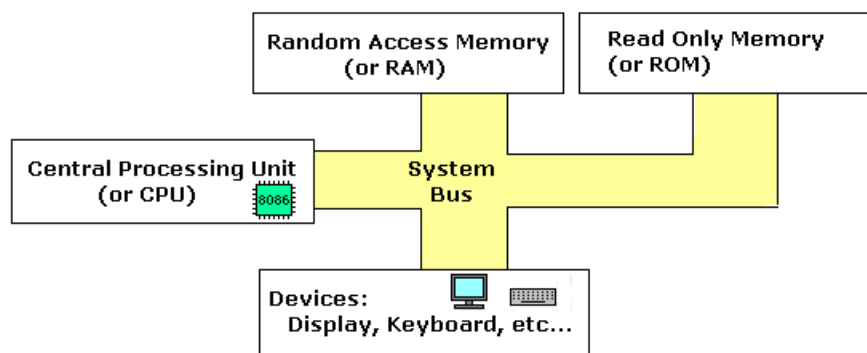


Fig. 1.1 Basic computer system containing a CPU, memory and I/O interfaces.

1.2 8086 as a CPU

The Intel 8086 is a comprehensive CPU whose derivatives are resident in modern personal computers (PC). 8086 is a 16-bit microprocessor with 20-bit address capability. The CPU functions as a two task processor, in what, one part manages the addressing and other part does all the computations and processing. The

functional part that computes and manages memory interactions is called Bus Interface Unit (BIU). The main computations and controls are all done by the execution unit (EU).

1.2.1 Bus Interface Unit

The BIU contains the 16-bit segment registers (CS, DS, SS and ES) and the instruction pointer (IP). It also contains a 6-byte queue arranged in first-in-first-out (FIFO) mode. The BIU contains the address-arithmetic-unit (AAU) that computes the 20-bit physical address from the segment registers and 16-bit offset.

20-bit physical address = [CS or DS or SS or ES]*16 + Offset.

1.2.2 Execution Unit

The EU contains four 16-bit general purpose registers (AX, BX, CX, DX), two 16-bit index registers (SI and DI) and two 16-bit pointer registers (BP and SP). It also contains the arithmetic and logic unit (ALU) that can perform 16-bit arithmetic and logical operations. There is a 16-bit status/flag register containing 9-bit flags that shows the salient status/state of the processor. The instruction codes coming from the queue of the BIU are decoded by the instruction decoder circuit.

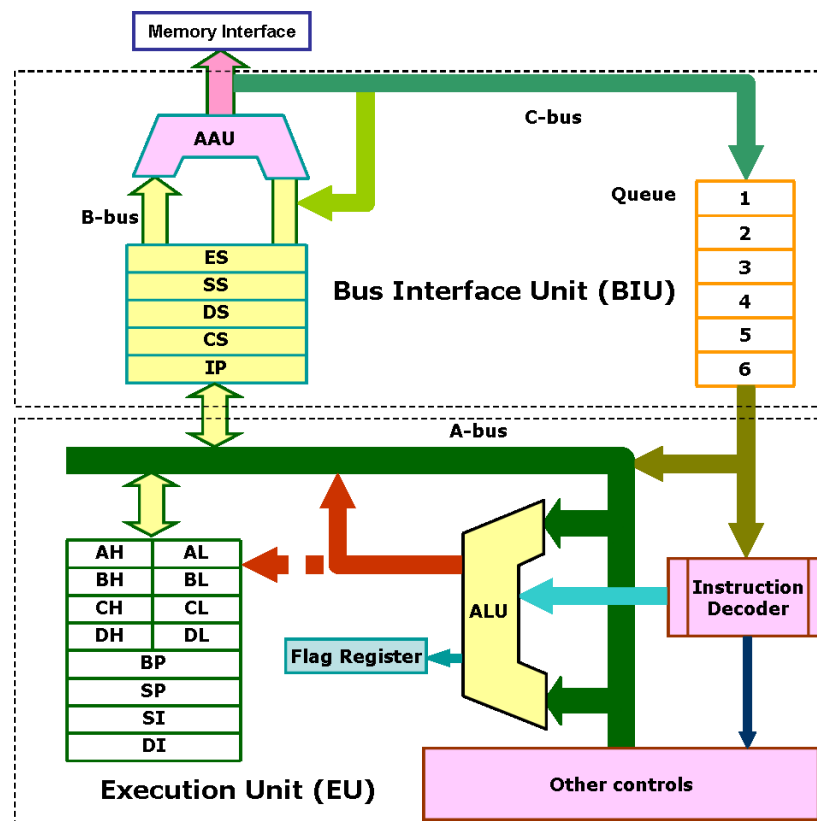


Fig. 1.2 The internal architecture of 8086 containing bus interface unit (BIU) and execution unit (EU).

1.3 General Purpose Registers

8086 CPU has 8 general purpose registers having names as follows

- AX** - the accumulator register (divided into **AH / AL**)
- BX** - the base address register (divided into **BH / BL**)
- CX** - the count register (divided into **CH / CL**)
- DX** - the data register (divided into **DH / DL**)
- SI** - source index register
- DI** - destination index register

BP - base pointer

SP - stack pointer

Accumulator (AX → AH+AL) register consists of 2 8-bit registers AL and AH, which can be combined together and used as a 16-bit register AX. AL in this case contains the low-order byte of the word, and AH contains the high-order byte. Accumulator can be used for I/O operations and string manipulation.

Base (BX → BH + BL) register consists of 2 8-bit registers BL and BH, which can be combined together and used as a 16-bit register BX. BL in this case contains the low-order byte of the word, and BH contains the high-order byte. BX register usually contains a data pointer used for based, based indexed or register indirect addressing.

Count (CX → CH + CL) register consists of 2 8-bit registers CL and CH, which can be combined together and used as a 16-bit register CX. When combined, CL register contains the low-order byte of the word, and CH contains the high-order byte. Count register can be used as a counter in string manipulation and shift/rotate instructions.

Data (DX → DH + DL) register consists of 2 8-bit registers DL and DH, which can be combined together and used as a 16-bit register DX. When combined, DL register contains the low-order byte of the word, and DH contains the high-order byte. Data register can be used as a port number in I/O operations. In integer 32-bit multiply and divide instruction the DX register contains high-order word of the initial or resulting number.

Instruction Pointer (IP) is a 16-bit register. This register has only one function - to point to the next instruction to be fetched to the BIU. Register IP is physically part of the BIU and not under direct control of the programmer as are the other pointer registers.

The following registers are both general purpose and index registers:

Stack Pointer (SP) is a 16-bit register pointing to program stack.

Base Pointer (BP) is a 16-bit register pointing to data in stack segment. BP register is usually used for based, based indexed or register indirect addressing.

Source Index (SI) is a 16-bit register. SI is used for indexed, based indexed and register indirect addressing, as well as a source data addresses in string manipulation instructions.

Destination Index (DI) is a 16-bit register. DI is used for indexed, based indexed and register indirect addressing, as well as a destination data addresses in string manipulation instructions.

Because registers are located inside the CPU, they are much faster than memory. Accessing a memory location requires the use of a system bus, so it takes much longer time. Accessing data in a register usually takes little time.

1.3.1 Segment registers

There are four different 64 kb memory segments for instructions, stack, data and extra data. The processor uses four segment registers to specify segment locations in the 1 MB memory space. The segment registers hold the base of each segment.

Code segment (CS) is a 16-bit register containing address of 64 kb segment with processor instructions. The processor uses CS segment for all accesses to instructions referenced by instruction pointer (IP) register. CS register cannot be changed directly. The CS register is automatically updated during far jump, far call and far return instructions.

Stack segment (SS) is a 16-bit register containing address of 64KB segment with program stack. By default, the processor assumes that all data referenced by the stack pointer (SP) and base pointer (BP) registers is located in the stack segment. SS register can be changed directly using POP instruction.

Data segment (DS) is a 16-bit register containing address of 64KB segment with program data. By default, the processor assumes that all data referenced by general registers (AX, BX, CX, DX) and index register (SI, DI) is located in the data segment. DS register can be changed directly using POP and LDS instructions.

Extra segment (ES) is a 16-bit register containing address of 64KB segment, usually with program data. By default, the processor assumes that the DI register references the ES segment in string manipulation instructions. ES register can be changed directly using POP and LES instructions.

1.4 Logical and Physical Addresses

Addresses within a segment can range from address 00000h to address 0FFFFh. This corresponds to the 64K-byte length of the segment.

An address within a segment is called an offset or logical address. A logical address gives the displacement from the address base of the segment to the desired location within it, as opposed to its "real" address, which maps directly anywhere into the 1 MB memory space. This "real" address is called the physical address.

1.4.1 Difference between the physical and the logical address

The physical address is 20 bits long and corresponds to the actual binary code output by the BIU on the address bus lines. The logical address is an offset from location 0 of a given segment

Although it is possible to store any data in the segment registers, this is never a good idea. The segment registers have a very special purpose - pointing at accessible blocks of memory. Segment registers work together with general purpose register to access any memory value. For example if we would like to access memory at the physical address **12345h** (hexadecimal), we should set the **DS = 1230h** and **SI = 0045h**. This is good, since this way we can access much more memory than with a single register that is limited to 16 bit values. CPU makes a calculation of physical address by multiplying the segment register by 10h and adding general purpose register to it ($1230h * 10h + 45h = 12345h$).

The address formed with 2 registers is called an **effective address**. By default **BX**, **SI** and **DI** registers work with **DS** segment register; **BP** and **SP** work with **SS** segment register. Other general purpose registers cannot form an effective address. Although **BX** can form an effective address, **BH** and **BL** cannot.

1.5 Flag Register

The 8086 contains a 16-bit flag register that indicates the salient states and status of the processor after execution of an instruction. The flag register is also called Status Register (**SR**) or Program Status Word (**PSW**). Each bit of the flag register is called a **flag** and can take a value of **1** or **0**. There are altogether nine flags in 8086. The bit assignment of the flag register is shown in Table 1.1.

Table 1.1 Flag register in 8086.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
				OF	DF	IF	TF	SF	ZF		AF		PF		CF

Carry Flag (CF) - this flag is set to **1** when there is an **unsigned overflow**. For example when you add bytes **255 + 1** (result is not in range 0...255). When there is no overflow this flag is set to **0**.

Zero Flag (ZF) - set to **1** when result is **zero**. For none zero result this flag is set to **0**.

Sign Flag (SF) - set to **1** when result is **negative**. When result is **positive** it is set to **0**. Actually this flag take the value of the most significant bit.

Overflow Flag (OF) - set to **1** when there is a **signed overflow**. For example, when you add bytes **100 + 50** (result is not in range -128...127).

Parity Flag (PF) - this flag is set to **1** when there is even number of one bits in the result, and to **0** when there is odd number of one bits. Even if result is a word (16-bit wide) only 8 low bits are analyzed.

Auxiliary Flag (AF) - set to **1** when there is an **unsigned overflow** for low nibble (4 bits).

Interrupt enable Flag (IF) - when this flag is set to **1**, the processor reacts to interrupts from external devices.

Direction Flag (DF) - this flag is used by string instructions to process data chains, when this flag is set to **0** - the processing is done forward, when this flag is set to **1** the processing is done backward.

1.6 8086 Interrupts

Interrupt operations fall into two classes; software or hardware initiated. The software initiated interrupts and software aspects of hardware interrupts are specified in the Instruction Set description. Hardware interrupts can be classified as non-Maskable or Maskable.

Interrupts result in a transfer of control to a new program location. A 256-element table containing address pointers to the interrupt service program locations resides in absolute locations 0 through 3FFH, which are reserved for this purpose.

Each element in the table is 4 bytes in size and corresponds to an interrupt type. An interrupting device supplies an 8-bit type number, during the interrupt acknowledge sequence, which is used to vector through the appropriate element to the new interrupt service program location.

For obtaining an interrupt address vector, the 8086 calculates two addresses in the pointer table where IP and CS are stored for a particular interrupt type.

Table 1.2 8086 Interrupt Pointer Table.

Interrupt Type Code		20-bit Memory Address	
0	IP	00001H	00000H
	CS	00003H	00002H
1	IP	00005H	00004H
	CS	00007H	00006H
2	IP	00009H	00008H
	CS	0000BH	0000AH
.	.	.	.
.	.	.	.
.	.	.	.
255	IP	003FDH	003FCH
	CS	003FFH	003FEH

For example, for the interrupt type "nn" (INT nn), the table address for IP = 4*nn and the table address for CS = 4*nn + 2. For servicing the 8086's nonmaskable interrupt (NMI), the 8086 assigns the type code 2 to this interrupt. The 8086 automatically executes the INT 2 instruction internally to obtain the interrupt address vector as

$$IP = 4*2 = 00008H$$

$$CS = 4 * 2 + 2 = 0000AH$$

The 8086 loads the values of IP and CS from the 20-bit physical address 00008H and 0000AH from the pointer table. The user must store the desired 16-bit values of IP and CS in these locations.

For example if the NMI service routine is in location CS:IP = 4000H:0100H, the user should store 4000H in location 00008H and 0100H in location 0000AH.

In response to an interrupt the 8086 does the following tasks:

1. Pushes the Flag register (PSW), CS and IP onto the stack
2. Clears the TF and IF flags
3. Loads the IP and CS from the pointer table using the type code

An Interrupt Service Routine (ISR) should always be terminated with the IRET (Interrupt Return) instruction. With the IRET instruction 8086 pops the top three stack words into IP, CS and PSW (flags), thus returning to the right place in the main program.

1.6.1 Interrupt Type Codes

The 256 interrupt type codes are assigned as follows:

- ❖ Predefined Interrupts (Types 0 to 4), 5 interrupts
- ❖ Reserved Interrupts (Types 5 to 31), 27 interrupts
- ❖ User defined Maskable Interrupts (Types 32 to 255), 224 interrupts

1.6.2 Predefined Interrupts

The five different predefined interrupts in 8086 are assigned the lowest locations in the interrupt pointer table. Table 1.3 shows the functional types of the predefined interrupts and their vector locations.

Table 1.3 Predefined interrupts and vector locations.

Interrupt Name	Type	Vector Address (CS:IP)
Divide by Zero Interrupt	Type 0	0002H:0000H
Single Step Interrupt	Type 1	0006H:0004H
NMI	Type 2	000AH:0008H
Breakpoint Interrupt	Type 3	000EH:000CH
Overflow Interrupt	Type 4	0012H:0010H

Divide by Zero Interrupt

The 8086 is automatically interrupted when a divide by zero is attempted. This interrupt is nonmaskable and is implemented by Intel as part of the execution of the divide instruction.

Single Step Interrupt

When the TF (TRAP flag, bit position of TF is D8 in the flag register) is set by an instruction the 8086 goes into single step mode. The TF can be set by the following instructions:

```
PUSHF                ;Save Flags
MOV BP, SP           ;[SP] → [BP]
OR [BP + 0], 0100H   ; Set TF = 1
POPF                 ; Pop flags
```

Once the TF is set to 1, 8086 automatically generates an interrupt of TYPE 1 after execution of each instruction. The user can write a service routine at the interrupt address vector to display memory locations, register contents etc to debug a

program. Single Step is nonmaskable and cannot be set by STI or cleared by CLI instructions.

NMI

The processor provides a single non-Maskable interrupt pin (NMI, pin 17), which has higher priority than the Maskable interrupt request pin (INTR). A typical use would be to activate a power failure routine. The NMI is edge-triggered on a LOW-to-HIGH transition. The activation of this pin causes a type 2 interrupt.

NMI is required to have duration in the HIGH state of greater than two CLK cycles, but is not required to be synchronized to the clock. Any high-going transition of NMI is latched on-chip and will be serviced at the end of the current instruction or between whole moves of a block-type instruction. Worst-case response to NMI would be for multiply, divide, and variable shift instructions. There is no specification on the occurrence of the low-going edge; it may occur before, during, or after the servicing of NMI. Another high-going edge triggers another response if it occurs after the start of the NMI procedure. The signal must be free of logical spikes in general and be free of bounces on the low-going edge to avoid triggering extraneous responses.

Breakpoint Interrupt

For inserting breakpoints into a program, the user can insert the instruction INT3 by replacing an instruction. Breakpoints are useful for program debugging.

Overflow Interrupt

This interrupt occurs if the overflow flag is set ($OF = 1$) and the INTO instruction is executed. For example the overflow flag is affected with the signed multiplication (MULS). The user can execute the INTO instruction after the MULS. If there is an overflow, an error service routine written by the user at the Type 4 interrupt address vector is executed.

1.6.3 Hardware Maskable Interrupts Via INTR

The 8086 provides a single interrupt request input (INTR) which can be masked internally by software with the resetting of the interrupt enable FLAG status bit. The interrupt request signal is level triggered.

It is internally synchronized during each clock cycle on the high-going edge of CLK. To be responded to, INTR must be present (HIGH) during the clock period preceding the end of the current instruction or the end of a whole move for a block type instruction. During the interrupt response sequence further interrupts are disabled. The enable bit is reset as part of the response to any interrupt (INTR, NMI, software interrupt or single-step), although the FLAGS register, which is automatically pushed onto the stack, reflects the state of the processor prior to the interrupt. Until the old FLAGS register is restored the enable bit will be zero unless specifically set by an instruction.

During the response sequence the processor executes two successive (back-to-back) interrupt acknowledge cycles. The 8086 emits the LOCK signal from T2 of the first bus cycle until T2 of the second. A local bus "hold" request will not be honored until the end of the second bus cycle. In the second bus cycle a byte is fetched from the external interrupt system (e.g., 8259A PIC), which identifies the source (type) of the interrupt. This byte is multiplied by four and used as a pointer into the interrupt vector lookup table (see Table 1.2). An INTR signal left HIGH will be continually responded to within the limitations of the enable bit and sample period. The INTERRUPT RETURN instruction includes a FLAGS pop, which returns the status of the original interrupt enable bit when it restores the FLAGS. A typical interrupt hardware responded through INTR is shown in Fig. 1.4. The interrupt code is placed at the input of 74LS244 and HIGH pulse is applied to the INTR pin. The INTA# signal enables the 74LS244 and transmits the interrupt code "nn" into the data bus.

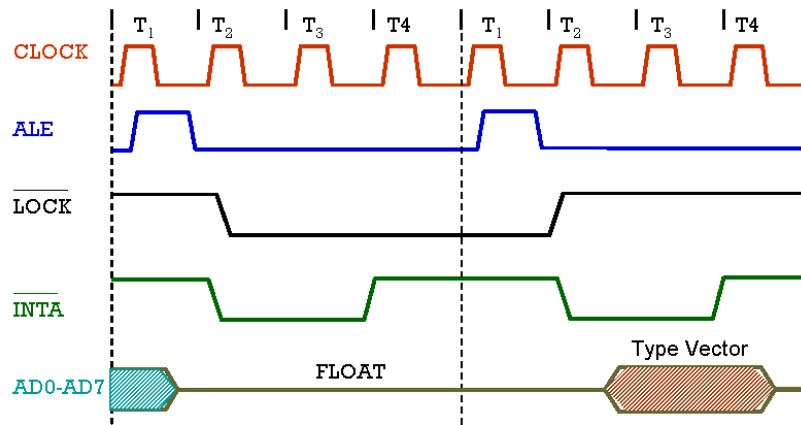


Fig. 1.3 8086 response signals for an interrupt request through INTR pin.

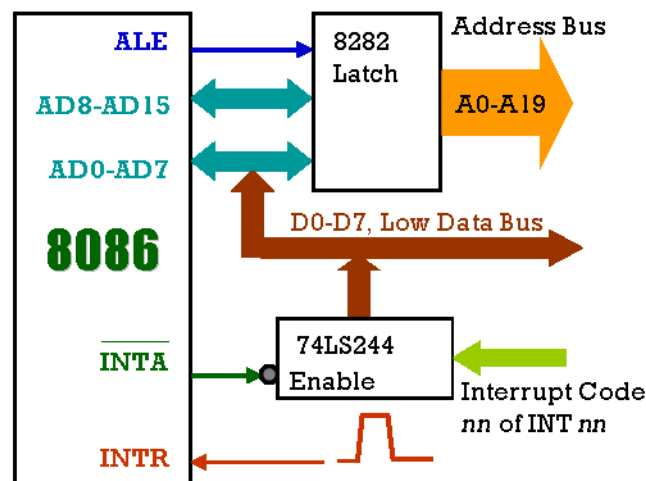


Fig. 1.4 Conceptual circuit of providing interrupt type code to 8086 for interrupt through INTR.

Review Questions

1. What are the functional components of 8086?
2. Name the registers available in 8086.
3. Discuss the special functions of the general purpose registers in 8086.
4. What are the functions of Bus Interface Unit (BIU)?
5. Differentiate between logical address and physical address in conjunction with 8086.
6. How 8086 does computes the 20-bit physical address from the 16-bit segment base and logical address?
7. What is pipelining? How does pipelining enhance computation speed of 8086?
8. Explain the structure and functionalities of the queue of 8086.
9. What is a stack? Explain the stack structure and stack operations of 8086.
10. What is a flag register? Explain the different flags available in the flag register of 8086.
11. Explain the interrupt structure in 8086.
12. Explain with suitable diagram how 8086 services hardware interrupt at its INTR pin.
13. What are the reserved interrupts in 8086?
14. Discuss the pre-defined interrupts in 8086.